CSIT970 Security Essentials

Week 3 – Cryptographic Issues

Dr. Steven Duong

School of Computing and Information Technology

UNIVERSITY
OF WOLLONGONG
AUSTRALIA

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Cryptography

- It is an inevitable line of defence for many systems and applications.
- It does not solve all the problems but many.
- Essential knowledge IT personnel should gain.
  - It is impossible to know cryptography in every detail,
  - but it's important to understand it (concept-application) correctly.

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Timeline in Cryptography

Cryptography ≠ Encryption + Decryption

1976

Symmetric Key Encryption

Key Exchange
Public Key Encryption
Digital Signature
Cryptographic Hash function
Message Authentication Code

UNIVERSITY OF WOLLONGONG AUSTRALIA

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Taxonomy of Cryptography

- Cryptography
  - Key-Based Cryptography
    - Symmetric
      - Symmetric Key Encryption
      - Message Authentication Code
    - Asymmetric
      - Key Exchange
      - Public Key Encryption
      - Digital Signature
  - Keyless Cryptography
    - Cryptographic Hash function

UNIVERSITY OF WOLLONGONG AUSTRALIA

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Taxonomy of Cryptography

Suppose a cryptography is used between Alice and Bob.

- If this cryptography requires Alice and Bob to share **the same secret key** first, it is called symmetric cryptography
- Otherwise, it is called asymmetric cryptography

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Key in Cryptography

In Cryptography, a key is usually represented with n-bit string

Once the key is known by the adversary, it can immediately break the scheme.

When the n-bit key is randomly chosen, the probability of guessing it is $\frac{1}{2^n}$.

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Key Size?

- Calculation: To find a random key by brute-force
  - n: key size (in bits)
  - c: CPU speed (Usually GHz/s) → Clock cycles (cc) per second
  - Time taking for brute-force attack: $t = 2^n/c$ (Assumption: it takes one clock cycle (cc) to perform one instance of search)
  - Example

    60-bit key searched by a computer with i5 3GHz processor, i.e. $c = 3 * 10^9\ cc/s$

    $t = 2^{60}/(3*10^9)\ s$ = 384,307,168 seconds = 160,752 hours = 4,448 days = 12 years

    60-bit key searched by a super computer with $c = 2*10^{16}\ cc/s$

    $t = 2^{60}/(2*10^{16})\ s$ = 58 seconds!

UNIVERSITY OF WOLLONGONG AUSTRALIA

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Key Size?

- Then, how about other key size?
  - ➢Example

    128-bit key, searched by a super computer with $c = 2*10^{16}$ cc/s

    $t = 2^{128}/(2*10^{16})$ s = 2^{49} (= 10^{15}) years! (Compare this number with any large number in https://en.wikipedia.org/wiki/Large_numbers)

    Hence n=128 is very safe
- Due to limitations from fundamental physics, there is no expectation that any digital computer (or combination) will be capable of breaking 256-bit random string via a brute-force attack.

UNIVERSITY OF WOLLONGONG AUSTRALIA

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Symmetric (Key) Encryption

- A sender and a receiver use the same secret key *k*.
  - *C = Enc(k, M)* *C*: *Ciphertext;* *M: Plaintext (or Message)*
  - *M =Dec(k, C)*
- Stream ciphers
  - Basically, each bit of plaintext is XOR-ed with a key stream.
  - It is fast and often used as "an encryption scheme on the fly".
- Block ciphers (such as AES)
  - Plaintext is divided into blocks of pre-defined size (64 bits, 128 bits etc.) and each block is encrypted using a symmetric cipher called "block cipher".

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Confidentiality (Secrecy) of Encryption

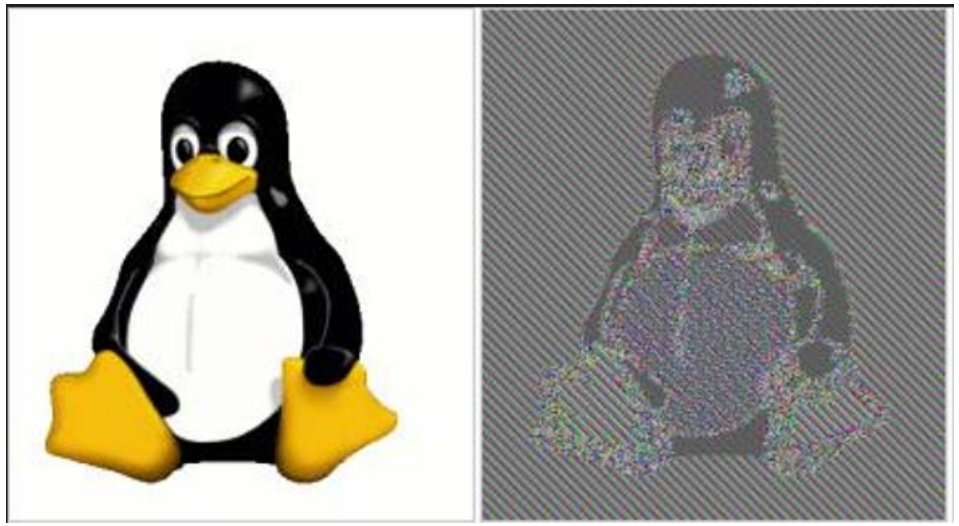
- Confidentiality (secrecy)
  - We assume that the plaintext that Alice sent to Bob is unknown to the adversary.
  - However, the adversary does have the ciphertext.
  - The adversary should not get anything useful information about the plaintext from the ciphertext.
    - ✓ This is a very information definition from the theoretical cryptographer's point of view, though.

Adversary

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Confidentiality (Secrecy) in Cryptography

- Look at the following picture. Do you think this is good encryption?



plaintext ciphertext

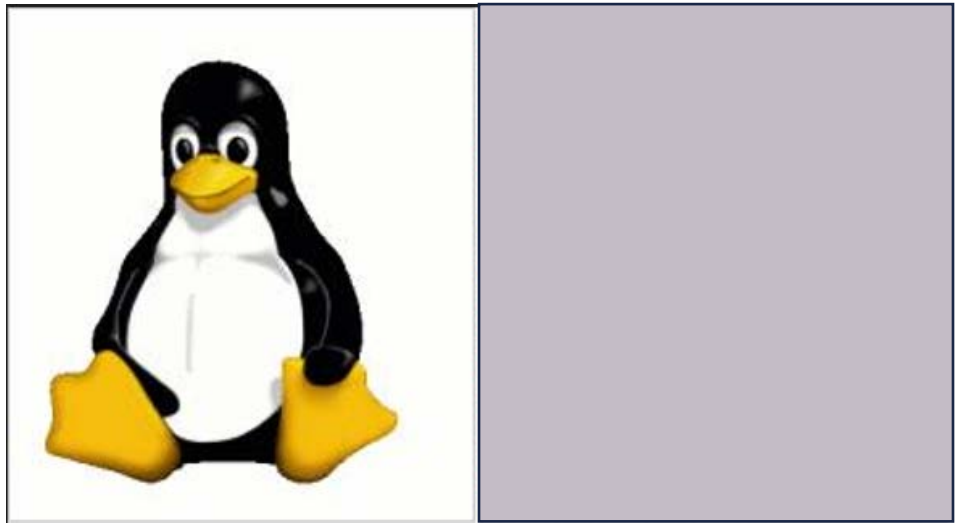
Adversary

UNIVERSITY OF WOLLONGONG AUSTRALIA

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Confidentiality (Secrecy) in Cryptography

- We want something like this…



plaintext ciphertext

Adversary

Looks like easy, but many encryption algorithms fail to achieve this!

UNIVERSITY OF WOLLONGONG AUSTRALIA

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Public Key Encryption

- Symmetric key encryption is efficient, but Alice and Bob might not have a shared secret key before.
- Public key encryption aims to achieve confidentiality between two users who do *not share a secret in advance*.
- The concept of public key encryption was proposed by Diffie and Hellman (in 1976) and the first public key encryption scheme, which is known as RSA, was introduced by **R**ivest, **S**hamir and **A**dleman.

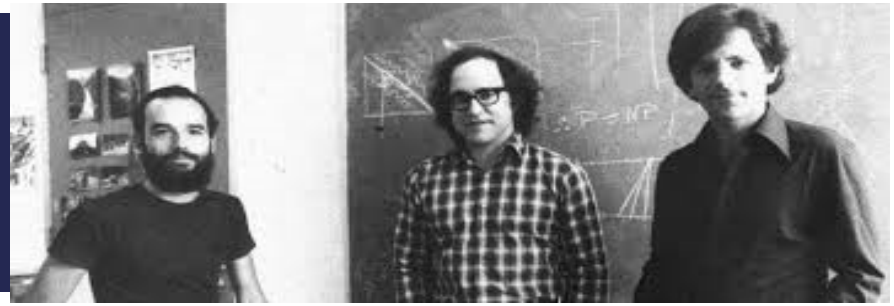
This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Public Key Encryption

- Notation: *pk*= public key, *sk*=secret key: *pk* is a key that can be published: *sk* is a secret key that the owner must keep secret
- In a public key encryption scheme, each user is assumed to generate a key pair (*pk*, *sk*).
  - The sender uses *pk* to encrypt her (plaintext) message *M*: *C* =Enc(*pk*, *M*)
  - The receiver uses *sk* to decrypt C: *M* = Dec(*sk*, *C*)
- Security: The most basic security property is that from *pk*, no one can come up with *sk*. Also, without having *sk*, no one will be able to get *M* from *C*. Usually these properties are guaranteed by the computationally hard problems. For this, *sk* and *pk* must be large!

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Public Key Encryption



- Example: RSA
  - $pk = (N,e)$, where $N=pq$, p & q primes; $sk=d$, where $ed=1 \bmod (p-1)(q-1)$
    - ✓ Hard to get d from (N, e): To get d, one needs to calculate $(p-1)(q-1)$. Without knowing the prime factors p and q, it seems hard to calculate $(p-1)(q-1)$.
    - ✓ Factoring N into p and q is known to be hard → Prime factorisation problem.
  - $\mathrm{Enc}(pk, M) = M^e \pmod N$
    - ✓ Hard to get M from C without knowing d: To get d, one needs to calculate $(p-1)(q-1)$. Again, factorisation problem.

UNIVERSITY OF WOLLONGONG AUSTRALIA

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Integrity and Authentication in Cryptography

- Integrity
  - We assume that the message is **known** to the adversary.
  - The adversary wants to alter (modify) the message in transit.
  - The receiver of the message should detect something has been altered.
- Authentication
  - We assume that the message is **known** to the adversary.
  - The adversary wants to alter (modify) the message and claims that the message was originated from a specific person, say Alice.

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Cryptographic Hash Functions

- Cryptographic hash functions are applied to guarantee the integrity of data. It takes the arbitrary length inputs and outputs a fixed length value.
- In particular, *cryptographic* hash function H satisfy the following property:
  - Preimage Resistant: It must be one way. Given *v* such that H(d)= *v*, finding value of d is infeasible.
  - Collision Resistant: It is infeasible to find the two distinct d and d' such that

$$H(d) = H(d')$$

UNIVERSITY OF WOLLONGONG AUSTRALIA

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Cryptographic Hash Functions

- It is often used as a component of other cryptographic algorithms.
  - For example, hash functions are used to compress a large data to a smaller one which can be taken as a message of digital signature for the efficiency purpose.
  - It also can be used to generated a deterministic random values for a given value.
- MD5, SHA1, SHA2 and SHA3 are cryptographic hash functions.
  - MD5 and SHA1 are not recommended any more for secure implementations. (They are broken!)

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Message Authentication codes

- Message Authentication Codes (MACs) are symmetric cryptography. A MAC, sometimes known as a tag, is a short piece of information used for authenticating a message.
  - t = MAC(k, M), where k is a key and t is called a "tag" or "MAC".
  - {0,1} = Verify(k, t, M)
  - Someone who does not know the key cannot compute a valid tag.
  - Someone who does not know the key cannot verify a tag.

- Producing MACs from hash functions
  - Using collision-free hash function H, compute t = H(k,M).

UNIVERSITY OF WOLLONGONG AUSTRALIA

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Message Authentication codes

- When MACs are used between Alice and Bob, they need to have a secret key k first.
- Digital Signatures
  - ➢message integrity – the message has not been altered in transit,
  - ➢message authentication – the message was sent by Alice,
  - ➢*non-repudiation – Alice cannot claim she did not send the message.*
- MACs
  - ➢message integrity – the message has not been altered in transit,
  - ➢message authentication – the message was sent by Alice,
- A signature can only be generated by Alice (Signer), but a tag can be generated by both Alice and Bob.

UNIVERSITY OF WOLLONGONG AUSTRALIA

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Digital Signature

- A signer has a key pair (pk,sk).
- In digital signature, the signer signs a message with her signing key and the resulting signature is verified using the signer's public key:
  - Signing key *sk* is private (secret).
  - Verification key *pk* is public.
- Digital Signature scheme
  - s = Sign(sk, M) s: <u>*Signature*</u>
  - {"Invalid (0)" or "Valid (1)"} = Verify(pk, M, s)

UNIVERSITY OF WOLLONGONG AUSTRALIA

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Digital Signature

- The output of digital signature provides a guarantee of three important security properties:
  - ➢message integrity – the message has not been altered in transit,
  - ➢message authentication – the message was sent by Alice,
  - ➢*non-repudiation – Alice cannot claim she did not send the message.*

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Public Key Infrastructure (PKI)

- In public key cryptography, a public key looks like this:

30 82 01 0a 02 82 01 01 00 dd 9e e4 f6 88 b5 0d d7 a1 5f bc 25 6e cc 44 14 cf 34 e2 b5 73
09 1e e4 4b 12 52 38 95 36 1a c6 66 ed f0 c8 03 c9 b3 43 45 4e 0d 6a 92 4b 1b eb 94 60 5b
11 b9 15 79 b1 a5 f6 fc 5d bf a4 30 59 84 02 dd 3f 6d 21 6a 44 b7 18 1c 24 fc f5 02 2e 87 0c
20 3e c6 c5 b6 9f ad 16 1b 76 86 e9 73 9c 8d 31 60 3a a0 f0 2f da ad 8e f6 74 c9 81 d3 ea
f7 5d ab 5d bb 05 63 b0 78 55 ed 72 13 a4 42 43 72 23 73 c0 de 33 9b 44 5c 89 a9 8a 90 d1
99 be bc f7 21 21 5f fb 22 8a 5c 50 b9 69 7c dc 87 92 ed 79 56 ed 32 55 41 9f af 41 f6 da d4
70 88 e9 a3 41 1d 66 9f a6 98 d2 7e 5b a9 52 38 1c 56 b4 cc 45 62 72 0c c7 f7 ef 2c 47 0a
3b 1a 7a ac e7 ae a9 a8 1e 98 43 b0 58 56 e9 41 44 72 e6 da 67 1c d7 b6 f4 e6 b4 90 5c b5
0a 98 b3 23 0c e7 35 6d 10 14 73 0e 94 5d 7c 4e 0a 18 f4 05 20 67 9f 02 03 01 00 01

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Public Key Infrastructure (PKI)

- In public key cryptography, public keys need to authenticated.
  - In other words, the relationship between the user's identity and the public key the user has created should be verified.
  - If this is not verified, there is a possibility that ciphertexts, signatures and other public key cryptographic objects are all bogus. (Man-in-the Middle attack)
- A widely-accepted solution for this is to have Bob prove to the Certification Authority (CA) that his identity ID_{Bob} and public key pk_{Bob} are legitimate. As a token of this verification, Bob will get something from the CA.

UNIVERSITY OF WOLLONGONG AUSTRALIA

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Public Key Infrastructure (PKI)

- That token is a digital signature generated as follows:
  - The CA signs Bob's ID with the public key that he has created:

$$s_{Bob} = \mathrm{Sign}(sk_{CA}, ID_{BOB} || pk_{Bob})$$

(That is, $ID_{BOB} || pk_{Bob}$ is regarded as a message input for the signature algorithm.)

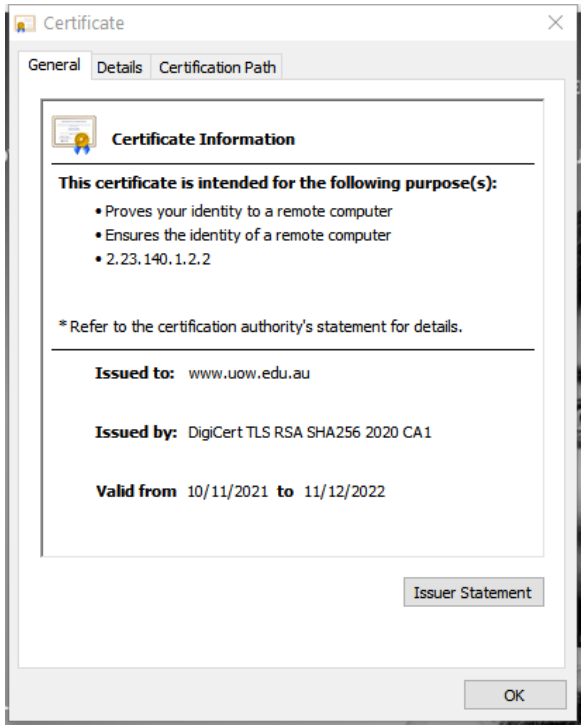
  - Alice (or anyone) who has obtained CA's verification (public) key pk_{CA} can check whether s_{Bob} is valid by running the verification algorithm. If it is valid, she can trust Bob's public key.

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Digital Certificate

- Digital certificate is a digital file that contains the user's ID and the public key he/she created and CA's signature on them.
- Other extra pieces of information such as
  - ➢Type of Signature
  - ➢Signature serial number
  - ➢CA's information
  - ➢Validity (such as expiry date of certificate)

  are contained in certificates.



UNIVERSITY OF WOLLONGONG AUSTRALIA

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Summary

- Each cryptography focuses on one specific security service.
- We might need to combine different cryptographies to have one security service protections.
- For example, Alice and Bob run Key Exchange to have a shared secret key before they run Symmetric-Key Encryption and Decryption.
- Asymmetric cryptography is much slower than symmetric cryptograpy.

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

The Ability of Cryptography

- The security provided by cryptography means that no adversary can break a cryptographic scheme within a well-defined and specific environment. (Usually, in this environment, we assume that the adversary does not know the key.)
- In the real world, it is impossible to have security in the universal environment.
  - For example, all cryptography schemes are insecure if keys are known by the adversary.

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

How to Bypass Cryptography Protections

- Passive Attacks (Implementation)
  - Ex) Side-channel attacks (Get the secret key)

- Active Attacks (Algorithm Substitution Attack)
  - Ex) Controlling key generation (Know the secret key)

  (Another issue is the key management and key theft)

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Side-Channel Attacks

In computer security, a side-channel attack is any attack based on information gained from the implementation of a computer system.

We can obtain the information of secret key via:

- Timing information (time cost of computing)
- Power consumption (off-peak, on-peak)
- Electromagnetic leaks
- Sound

(While cryptography assumes that the algorithms can be safely implemented)

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Side-Channel Attacks

- Example: RSA
  - $pk = (N,e)$, sk=d
  - Enc(pk, M) = M^e (mod N) =C
  - Dec(C, d)= C^d =M <u>(This is called exponentiation)</u>

Here, d is an n-bit string when decrypting the ciphertext.

UNIVERSITY OF WOLLONGONG AUSTRALIA

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

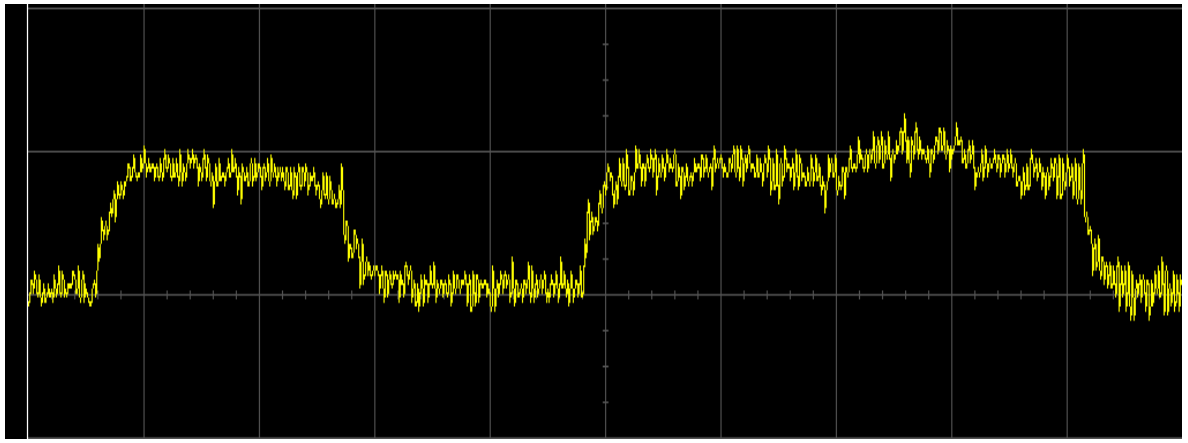
Side-Channel Attacks

ALGORITHM 1

LEFT-TO-RIGHT BINARY METHOD

Input:	X, N, $E = (e_{k-1}, \ldots, e_1, e_0)_2$
Output:	$X^E \bmod N$

```
R := 1;
for i = k − 1 downto 0 do
    R := R · R mod N;            — squaring
    if e_i = 1 then
        R := R · X mod N;        — multiplication
    end if
end for
return R
```

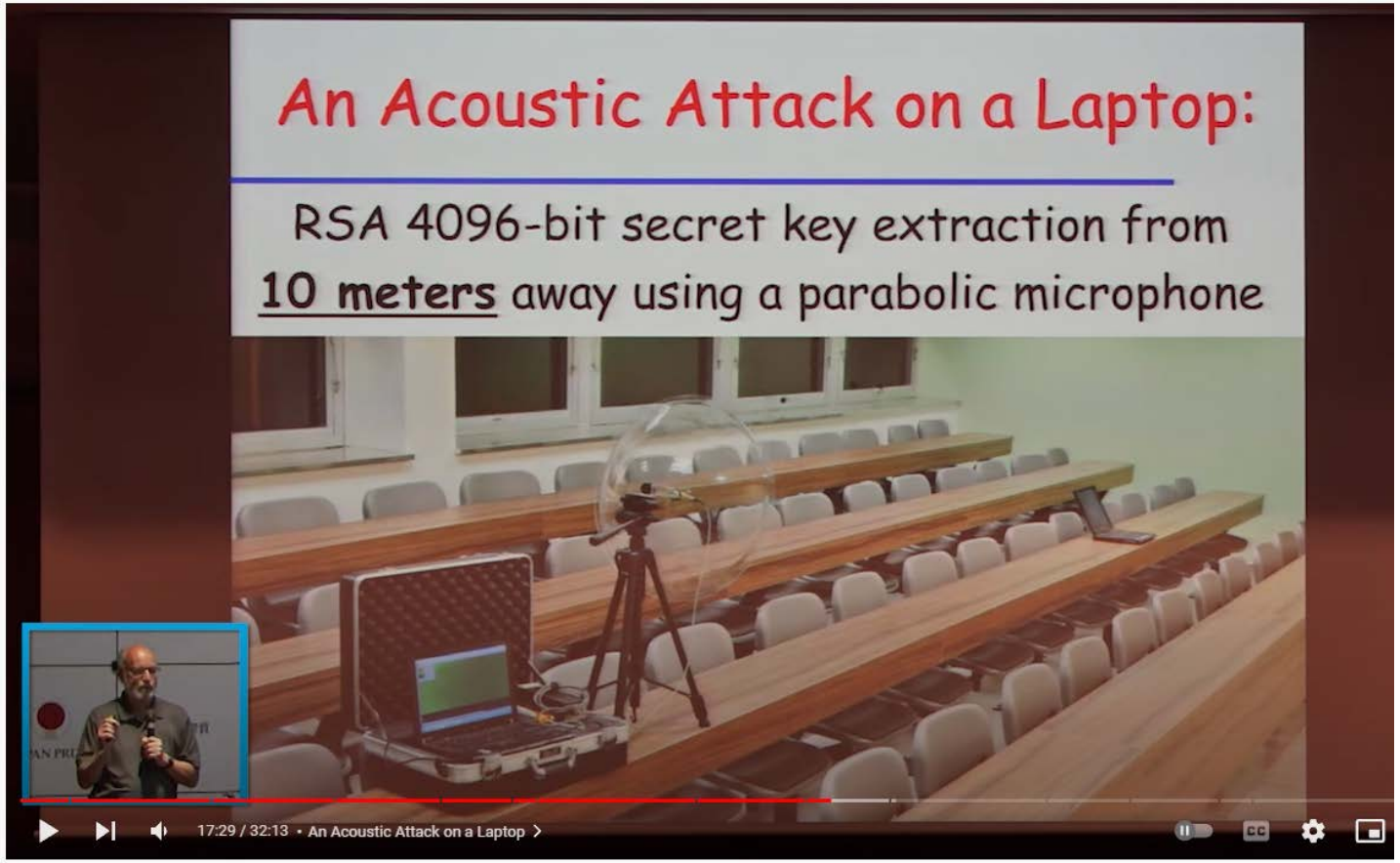


An attempt to decode RSA key bits using power analysis. The left peak represents the CPU power variations during the step of the algorithm without multiplication, the right (broader) peak – step with multiplication, allowing an attacker to read bits 0, 1. (From Wiki)

UNIVERSITY OF WOLLONGONG AUSTRALIA

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Side-Channel Attacks



2017 Japan Prize Commemorative Lecture: Dr. Adi Shamir

UNIVERSITY OF WOLLONGONG AUSTRALIA

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Algorithm Substitution Attack

- Roughly, the ASA (Algorithm Substitution Attack) enables an adversary, often called “big brother”, to replace some parts of a legitimate cryptographic scheme or protocol with a modified one.
- This is related to “mass surveillance” we have learned.
- The ASA wants to let computation results subliminally leak the crucial information about a secret key or a confidential message to the adversary.

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Controlling Key Generation

- In cryptography, we run a key generation algorithm to generate a key *k* for symmetric cryptography or a key pair (*pk*, *sk*) for public key cryptography.

- The first step of the key algorithm is to generate a random *n*-bit string *S* first, which will be set as *k* or *sk*.

- It is hard to know whether the string *S* is indeed randomly chosen, because it is hard to judge whether the random algorithm is compromised or not (unless we can check the code).

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Controlling Key Generation

- For example, two very special bit strings

00000000000000

11111111111111

could be generated from a good random number generation algorithm.

UNIVERSITY OF WOLLONGONG AUSTRALIA

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Controlling Key Generation

- Let `RandStr(n)` be the function that outputs a random n-bit string.
- We may want to call this function to generate a (random) secret key using the following function:

```
def KeyGen(n):
    A = RandStr(10)
    B = H(A)   #H is the hash function that outputs n-bit
    return H(A)
```

UNIVERSITY OF WOLLONGONG AUSTRALIA

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim

Controlling Key Generation

- What is the problem of this function?
  - As long as we can correctly guess the output of RandStr(10) (A=**10-bit random string)**, we can recover the whole key from the KeyGen( ) function.
  - We only need to produce 2^{10}=1024 strings and try them one by one (This is called a brute-force attack).

UNIVERSITY OF WOLLONGONG AUSTRALIA

This slide is copyrighted. It must not be distributed without permission from Joonsang Baek and Jongkil Kim